Regras BDD

Perfila — Sistema de Segmentação e Perfilamento do Comportamento do Cliente — Go

Equipe Omega · GoLang & CLP · Documento 3 de 3 · **Versão 1.0 (03/11/2026)**

Este documento define como a equipe escreve e executa os critérios de aceite em Behavior-Driven Development (BDD). As features em Gherkin derivadas da modelagem ficam em `features/*.feature` (seção 6 traz o índice). Referências: **Spec** é o Documento 1 (`specs/spec.md`), **Plano** é o Documento 2 (`specs/plan.md`) e **Constituição** é o `CLAUDE.md`. Os IDs UC-xx, RN-xx, C-xx e D-xx vêm da Spec.

1. Papel do BDD no SDD

Na Spec, cada regra de negócio descreve o comportamento em prosa. No BDD, a mesma regra vira um exemplo concreto, legível por pessoas e executável pelo CI. Assim a Spec fica verificável: um cenário vermelho significa que o código ou a Spec está errado, e um agente de IA tem um critério objetivo de "pronto". A Constituição (C-10) exige que toda regra de negócio e toda transição de estado tenha cenário.

2. Regras de escrita

ID	Regra
B-01	**Idioma e vocabulário.** Português do Brasil, com `# language: pt` na primeira linha. Usar os termos do glossário (seção 3). Cenários de domínio e aplicação não citam SQL, JSON, struct ou outros detalhes técnicos.
B-02	**Um comportamento por cenário.** O título declara a regra ("Revogar consentimento concedido"), não o teste ("Testar revogação").
B-03	**Estrutura Dado, Quando, Então.** Dado descreve o estado inicial, Quando traz uma única ação e Então traz o resultado observável. No máximo 7 passos por cenário; E e Mas apenas continuam o tipo anterior.
B-04	**Declarativo, não imperativo.** Descrever a intenção e a regra, não a sequência de chamadas ou cliques.
B-05	**Variações de dados em tabela.** Usar Esquema do Cenário com Exemplos e incluir sempre os valores-limite (por exemplo 29, 30 e 31 dias).
B-06	**Contexto só para o que é comum.** Pré-condições compartilhadas por todos os cenários da feature, até 5 passos.
B-07	**Tempo controlado.** Toda feature fixa a data atual em um passo. Proibido depender do relógio real (C-07).
B-08	**Dados sintéticos.** IDs como `c-001`, valores inventados. Nunca dados pessoais reais.
B-09	**Formato dos parâmetros.** Texto entre aspas duplas, números sem aspas, datas em ISO 8601 UTC.
B-10	**Cobertura mínima.** Cada RN, cada transição de estado (válida e inválida) e cada erro do catálogo da Spec 3.5 tem pelo menos um cenário. Cada UC tem fluxo feliz e pelo menos uma falha. Todo UC que processa dado pessoal tem cenário de consentimento inválido (`@lgpd`).
B-11	**Tags obrigatórias.** Pelo menos um `@UC-xx` ou `@RN-xx`, **exatamente uma** camada por cenário (`@dominio`, `@aplicacao`, `@api` ou `@implantacao`, herdada da feature ou declarada no cenário) e `@lgpd` quando aplicável. `@wip` nunca vai para a branch principal; `@slow` marca cenários que sobem contêineres.

ID	Regra
B-12	**Passos finos.** Os step definitions só traduzem texto em chamadas e asserções; nenhuma regra de negócio dentro de um passo. Reutilizar passos existentes antes de criar novos.
B-13	**Independência.** Nenhum cenário depende de outro; o estado inicial vem sempre do Dado.
B-14	**Cenário vermelho é defeito.** Mudar um cenário exige mudar a Spec no mesmo PR, com aprovação do Guardião do BDD. O agente de IA nunca edita `.feature` para fazê-lo passar (C-12).
B-15	**Cenário antes do código.** Em cada tarefa: vermelho, depois verde, depois refatoração.

3. Glossário ubíquo

Termo nos cenários	Identificador em Go
cliente	`Cliente`
consentimento	`ConsentimentoLGPD`
evento	`EventoComportamental`
perfil	`PerfilCliente`
segmento	`Segmento`
recomendação	`RecomendacaoProduto`
produto, catálogo	`Produto`, `ProdutoRepository`
coleta	`ColetarDadosUseCase`
processamento	Fluxo UC-01 a UC-04 pela API (RN-09)

4. Camadas de teste e tags

Tag	O que exercita	Dependências	Velocidade
`@dominio`	Entidades e serviços puros, com relógio fixo	Nenhuma	Milissegundos
`@aplicacao`	Casos de uso com ports em memória (fakes)	Fakes em memória	Milissegundos
`@api`	Handlers e middlewares via `httptest`	Fakes em memória	Milissegundos
(contrato, fora do BDD)	Os mesmos testes de contrato dos ports contra fake e PostgreSQL	PostgreSQL em contêiner	Segundos
`@implantacao` `@slow`	Nginx, duas réplicas e banco reais	Docker Compose	Minutos

5. Execução com godog

Ferramenta: godog (Cucumber para Go), que lê Gherkin em português. Os arquivos ficam em `features/` e os passos em `features/steps/`. O texto do passo no godog não inclui a palavra-chave (Dado, Quando, Então).

```
// features/bdd_test.go
const tagsPadrao = "~@implantacao && ~@wip" // Core e Support concluídos

func TestFeatures(t *testing.T) {
    tags := os.Getenv("BDD_TAGS")
    if tags == "" {
        tags = tagsPadrao
    }
    suite := godog.TestSuite{
        ScenarioInitializer: steps.InitializeScenario,
        Options: &godog.Options{
            Format: "progress", Paths: []string{"."}, Tags: tags,
            Strict: true, // passo indefinido reprova
            TestingT: t,
        },
    }
    if suite.Run() != 0 {
        t.Fatal("cenários BDD falharam")
    }
}

// features/steps/steps.go
func InitializeScenario(sc *godog.ScenarioContext) {
    m := novoMundo() // relógio fixo, fakes em memória e último resultado — um por cenário (B-
13)
    sc.Step(`^que a data atual é "([^"]*)"$`, m.dataAtual)
    sc.Step(`^que existe o cliente "([^"]*)" ativo$`, m.clienteAtivo)
    sc.Step(`^a operação falha com o erro "([^"]*)"$`, m.falhaComErro)
    // ...
}
```

- O alvo `make bdd` executa `go test ./features/...`; `make verify` inclui `make bdd`.
- Filtro por fase: `@implantacao` fica fora até a fase Generic. Para outra seleção: `BDD_TAGS="@lgpd" make bdd`. A sintaxe é a do godog (`~` nega, `&&` e `,` combinam).
- Identificadores sintéticos (`c-001`, `Livro A`) viram UUIDs determinísticos no passo (B-08).
- Cada passo reutilizável aparece uma vez em `features/steps/`; um passo novo exige justificativa no PR (B-12).

6. Features em Gherkin

As features vivem em `features/` e são a versão executável deste documento. Estado em 03/11/2026:

Arquivo	UC / regra	Camada	Cenários executados	Fase
`consentimento.feature`	UC-08, RN-02, RN-03	`@aplicacao`	13	Core ✅
`evento.feature`	UC-01, RN-01, RN-04	`@aplicacao`	17	Core ✅
`perfilamento.feature`	UC-02, RN-05	`@aplicacao`	9	Core ✅
`segmentacao.feature`	UC-03, RN-06	`@aplicacao` e `@dominio`	19	Core ✅
`recomendacao.feature`	UC-04, RN-07, RN-08	`@aplicacao`	6	Core ✅
`recomendacao_ciclo.feature`	3.4.2, RN-08	`@dominio` e `@aplicacao`	9	Core ✅

Arquivo	UC / regra	Camada	Cenários executados	Fase
`cliente.feature`	UC-09 (D-22)	`@aplicacao`	6	Core ✅
`api.feature`	UC-01..04, UC-08, UC-09, RN-09	`@api`	20	Support ✅
`implantacao.feature`	UML8	`@implantacao` `@slow`	3	Generic ⬜

Total executado: **99 cenários, 668 passos, todos verdes** (Core 79 + Support 20).

6.1 Mudanças da v1.0 nas features (aprovadas junto com a Spec v1.0, B-14)

Feature	Mudança	Regra atendida
consentimento	Camada única `@aplicacao` (antes `@dominio` `@aplicacao` juntas)	B-11
consentimento	"A validade depende somente do estado" ganhou o passo **Quando** (consulta)	B-03
consentimento	Novos: sem registro equivale a NAO_CONCEDIDO; cliente inexistente; guarda violada não muda o estado	B-10 (UC-08 falha, `cliente_nao_encontrado`)
evento	Novo esquema com valores-limite aceitos (valor 0, compra 0.01, timestamp = agora); `navegacao` com valor negativo	B-05
evento	Cenário LGPD também verifica que nenhum evento é gravado	C-03
perfilamento	Passo explícito "o identificador do perfil é preservado"	RN-05 (upsert preserva ID)
segmentacao	Novo esquema de critérios válidos; exemplo `!=` (operador fora da gramática)	B-10, RN-06
recomendacao	Novo: perfil sem segmento → `perfil_nao_encontrado`; "sem categoria" também verifica que nada é gravado	B-10 (catálogo 3.5)
recomendacao_ciclo	Camada por cenário (`@dominio` × `@aplicacao` no comando); transição inválida verifica que o estado não muda	B-11
api, implantacao	Sem mudança de conteúdo; cabeçalho indica a fase	—